PCS5916 Computação de Alto Desempenho e Aplicações

Relatório dos Exercícios da Aula 03

André Moreno Goveia
13682785

4 de outubro de 2026

1 Ambiente de testes

Os testes rodaram em um processador de 8 núcleos e 16 threads. Compilei os programas com `gcc -fopenmp` usando `-O0` e `-O2` e rodei cada versão 10 vezes. Os tempos das tabelas são médias e medem só o trecho de cálculo, sem a inicialização dos dados. O speedup é o tempo sequencial dividido pelo tempo paralelo. Em todos os casos as versões paralelas deram os mesmos resultados das sequenciais.

2 Exercício 1: estatísticas de um vetor

O programa calcula a soma, o maior e o menor elemento e a quantidade de pares de um vetor com 100 milhões de inteiros positivos. Na versão sequencial, um único laço percorre o vetor e atualiza as quatro estatísticas ao mesmo tempo.

2.1 Versão com parallel for (ex1_for.c)

A ideia aqui é dividir o vetor. Com `#pragma omp parallel for num_threads(4)`, cada thread fica com um quarto dos elementos e calcula as quatro estatísticas na sua parte. Como todas as threads atualizam as mesmas variáveis, usei `reduction(+:soma, pares)`, `reduction(max:maior)` e `reduction(min:menor)`. Assim cada thread trabalha com uma cópia própria e o OpenMP junta os resultados no fim, sem precisar de região crítica.

2.2 Versão com parallel sections (ex1_sections.c)

Nesta versão a divisão é por tarefa. Criei quatro seções, uma para cada estatística, e cada thread percorre o vetor inteiro calculando só a sua. Cada seção escreve em uma variável diferente, então não há condição de corrida.

Na primeira tentativa as seções escreviam direto nas variáveis `soma`, `maior`, `menor` e `pares`. Essas variáveis ficam lado a lado na memória e caem na mesma linha de cache, e por isso cada escrita de uma thread invalidava a cache das outras. Esse falso compartilhamento deixou o programa com cerca de 0,82 s em `-O2`, mais de dez vezes mais lento que o sequencial. Para resolver, cada seção passou a acumular em uma variável local e só copia o resultado no final.

2.3 Resultados

Sem otimização, o `parallel for` chegou a 3,69, bem perto do ideal de 4, já que cada thread faz um quarto do trabalho e juntar as reduções custa quase nada. O `parallel sections` ficou em 1,15. Faz sentido, porque cada thread lê o vetor inteiro e o tempo total acaba sendo o da seção mais lenta. Além disso, essa abordagem não escala, porque com só quatro estatísticas uma quinta thread não teria o que fazer.

	-O0		-O2	
Versão	Tempo (s)	Speedup	Tempo (s)	Speedup
Sequencial (ex1_seq.c)	1,2089	1,00	0,0738	1,00
parallel for (ex1_for.c)	0,3275	3,69	0,0779	0,95
parallel sections (ex1_sections.c)	1,0544	1,15	0,0635	1,16

Tabela 1: Tempos médios de 10 execuções do exercício 1 com 4 threads.

Com `-O2` o cenário muda. O compilador vetoriza o laço e a versão sequencial lê os 400 MB do vetor em cerca de 74 ms. Nesse ponto o gargalo passa a ser a memória e não o processamento, então colocar mais threads não ajuda e o `parallel for` fica até um pouco mais lento por causa do custo de criar as threads. O `parallel sections` ganhou 16%, provavelmente porque cada seção tem um laço mais simples, que o compilador otimiza melhor. No fim, esse problema faz muito pouca conta por elemento lido, e o paralelismo só vale a pena quando o cálculo pesa mais que o acesso à memória.

3 Exercício 2: multiplicação de matrizes

Parti do `mat_mul.c` da aula 02, que multiplica duas matrizes 1000×1000 com três laços aninhados. Cada linha de C depende só de uma linha de A e da matriz B, então linhas diferentes podem ser calculadas em paralelo sem nenhuma sincronização.

3.1 Versão com parallel for (ex2_for.c)

Coloquei `#pragma omp parallel for num_threads(4) private(j, k)` no laço externo, e cada thread calcula 250 linhas de C. O `private(j, k)` é necessário porque no código original essas variáveis são declaradas no início do `main` e seriam compartilhadas entre as threads. O `i` já é privado por ser o índice do laço paralelizado.

3.2 Versão com parallel sections (ex2_sections.c)

Movi os três laços para uma função `multiplica(inicio, fim)` que calcula um intervalo de linhas de C. Cada uma das quatro seções chama essa função para um bloco de 250 linhas. Na prática é a mesma divisão que o `parallel for` faz, só que feita à mão. Como todas as linhas têm o mesmo custo, a carga fica equilibrada.

3.3 Resultados

	-O0		-O2	
Versão	Tempo (s)	Speedup	Tempo (s)	Speedup
Sequencial (mat_mul.c)	6,4245	1,00	0,6315	1,00
parallel for (ex2_for.c)	2,1242	3,02	0,2644	2,39
parallel sections (ex2_sections.c)	2,0383	3,15	0,2532	2,49

Tabela 2: Tempos médios de 10 execuções do exercício 2 com 4 threads.

As duas versões tiveram speedups parecidos, 3,02 e 3,15 sem otimização, o que era esperado já que dividem o trabalho do mesmo jeito. A diferença entre elas fica dentro da variação normal entre execuções. O speedup não chega a 4 principalmente por causa do acesso a $B[k][j]$, que percorre uma coluna e pula 4000 bytes a cada leitura, gerando muitas faltas de cache. Com

quatro threads essas faltas disputam a mesma cache L3 e a mesma memória. O processador também baixa a frequência quando mais núcleos estão ativos. Com `-O2` o cálculo fica cerca de dez vezes mais rápido, a memória passa a pesar mais e o speedup cai para cerca de 2,4. Mesmo assim o ganho é bem maior que no exercício 1, porque aqui são 10^9 multiplicações sobre apenas 12 MB de dados.

4 Exercício 3: jantar dos filósofos

Cinco filósofos sentam em volta de uma mesa com um garfo entre cada par, e cada um precisa dos dois garfos ao seu lado para comer. Nas duas versões, `#pragma omp parallel num_threads(5)` cria uma thread por filósofo. O filósofo i usa o garfo i à esquerda e o garfo $(i+1)$ mod 5 à direita. Cada um pensa por até 1 ms, pega os garfos, come por 0,5 ms e devolve os garfos. A simulação dura 3 s e, no final, o programa mostra quantas vezes cada filósofo comeu e há quanto tempo estava sem comer. Coloquei também uma pausa de 1 ms entre pegar o primeiro e o segundo garfo. Ela não muda a lógica, mas deixa os problemas de sincronização bem mais fáceis de observar.

4.1 Versão padrão (ex3_padrao.c)

Cada garfo é um inteiro do vetor `garfos`, que vale 1 quando está em uso. O filósofo fica em um laço esperando o garfo esquerdo ficar livre, marca como ocupado, faz o mesmo com o direito, come e solta os dois. O vetor é `volatile` para o compilador não guardar o valor em registrador, o que transformaria a espera em um laço infinito. Os laços de espera também olham o relógio, mas só para o programa conseguir terminar depois dos 3 s.

Essa versão tem três problemas. Primeiro, existe uma condição de corrida, porque testar o garfo e marcá-lo são dois passos separados. Dois vizinhos podem ver o mesmo garfo livre ao mesmo tempo e os dois pegarem.

Segundo, pode acontecer deadlock. Se todos pegam o garfo esquerdo ao mesmo tempo, cada um fica esperando o garfo que o vizinho da direita está segurando, e ninguém sai do lugar. Com a pausa de 1 ms isso aconteceu logo na primeira rodada em todas as execuções, e todos os filósofos terminaram sem comer nenhuma vez. Tirando a pausa, o deadlock continuou aparecendo, só que em momentos aleatórios, depois de 115 a 867 refeições por filósofo.

Terceiro, pode haver starvation. Nada garante que um filósofo consiga os dois garfos, então um que perde a disputa várias vezes seguidas pode ficar sem comer enquanto os outros comem. E como a espera é ativa, cada filósofo esperando ocupa um núcleo inteiro à toa.

4.2 Versão com locks (ex3_locks.c)

Aqui cada garfo é um `omp_lock_t`. O filósofo pega o garfo com `omp_set_lock`, que bloqueia a thread até o lock ficar livre, e devolve com `omp_unset_lock`. O OpenMP garante que só uma thread segura cada lock por vez, o que resolve a condição de corrida e acaba com a espera ativa.

Só trocar por locks não resolve o deadlock, porque se todos pegarem o garfo esquerdo primeiro o ciclo de espera continua igual. Para quebrar o ciclo, cada filósofo pega primeiro o garfo de menor número. Os filósofos 0 a 3 continuam começando pela esquerda, mas o filósofo 4 pega o garfo 0 antes do garfo 4. Com isso a espera circular não consegue mais se formar.

Os locks do OpenMP também não garantem ordem de chegada, então um filósofo ainda poderia perder a disputa repetidamente. Para evitar isso, adicionei uma regra simples. Antes de pegar os garfos, o filósofo confere se já comeu mais que algum vizinho e, se sim, espera os vizinhos o alcançarem. Os contadores usam `#pragma omp atomic` para leitura e escrita. Quem comeu menos nunca precisa esperar, por isso a mesa sempre avança. Como a espera acontece sem segurar garfo nenhum, ela também não cria um novo deadlock.

Nos testes, mesmo com a pausa de 1 ms, a versão com locks rodou os 3 s sem travar. Cada filósofo comeu entre 859 e 861 vezes e nenhum ficou mais de 2 ms sem comer no final.